

Assignment No. 7

Activity Log Analyzer using Python

ARITRA TALUKDAR

Roll No: CSB24034

March 13, 2026

1 Introduction

This assignment implements an Activity Log Analyzer using Python. The program generates dummy user activity logs and performs the following operations:

- Calculates total screen time per user using `reduce()`
- Identifies the top K most active users
- Extracts unique actions performed
- Measures execution time

The program demonstrates the use of advanced Python concepts such as type hints, list comprehension, dictionary operations, and functional programming.

2 Algorithm

1. Take input: number of users, number of logs, and value of K.
2. Generate dummy activity logs.
3. Compute total screen time per user using `reduce()`.
4. Sort users to find top K active users.
5. Extract unique actions using set comprehension.
6. Display formatted output.

3 Python Implementation

```
from typing import List, Dict, Set
from collections import defaultdict
from functools import reduce
import random
import time

# -----
# Function 1: Total Time Per User (Using reduce)
# -----
def total_time_per_user(logs: List[Dict[str, object]]) -> Dict[
    str, float]:
    """
    Returns total screen time per user using reduce().
    """

    def reducer(acc: Dict[str, float], log: Dict[str, object]) ->
        Dict[str, float]:
```

```

        user = log["user"]
        duration = log["duration"]
        acc[user] = acc.get(user, 0.0) + float(duration)
        return acc

    return reduce(reducer, logs, {})

# -----
# Function 2: Most Active Users (Top K)
# -----
def most_active_users(logs: List[Dict[str, object]], k: int) ->
    List[str]:
    """
    Returns top k users based on total activity time.
    """

    totals = total_time_per_user(logs)

    sorted_users = sorted(
        totals,
        key=lambda user: totals[user],
        reverse=True
    )

    return sorted_users[:k]

# -----
# Function 3: Unique Actions
# -----
def unique_actions(logs: List[Dict[str, object]]) -> Set[str]:
    """
    Returns unique set of actions using set comprehension.
    """
    return {log["action"] for log in logs}

# -----
# Generate Dummy Activity Logs
# -----
def generate_logs(num_users: int, num_logs: int) -> List[Dict[str,
    object]]:

    users = [f"ROLL_{1000+i}" for i in range(num_users)]

    actions = [
        "Instagram",
        "YouTube",
        "WhatsApp",
        "Chrome",

```

```

        "Facebook",
        "Netflix",
        "VSCode",
        "Telegram"
    ]

    logs = [
        {
            "user": random.choice(users),
            "action": random.choice(actions),
            "duration": round(random.uniform(1, 120), 2)
        }
        for _ in range(num_logs)
    ]

    return logs

# -----
# Pretty Printing Utilities
# -----
def print_separator():
    print("=" * 60)

def print_totals(totals: Dict[str, float]):
    print_separator()
    print("TOTAL SCREEN TIME PER USER")
    print_separator()

    for user, time_spent in sorted(
        totals.items(),
        key=lambda x: x[1],
        reverse=True):

        print(f"{user:<15} | {time_spent:>8.2f} minutes")

    print_separator()

def print_top_users(top_users: List[str], totals: Dict[str, float]):
    print("\nTOP ACTIVE USERS")
    print_separator()

    for rank, user in enumerate(top_users, start=1):
        print(f"{rank}. {user:<15} -> {totals[user]:.2f} minutes")

    print_separator()

```

```

def print_unique_actions(actions: Set[str]):
    print("\nUNIQUE ACTIONS PERFORMED")
    print_separator()

    for action in sorted(actions):
        print(f"- {action}")

    print_separator()

# -----
# MAIN EXECUTION
# -----
if __name__ == "__main__":

    print_separator()
    print("ACTIVITY LOG ANALYZER")
    print_separator()

    num_users = int(input("Enter number of users: "))
    num_logs = int(input("Enter number of activity records: "))
    k = int(input("Enter value of K (Top K users): "))

    logs = generate_logs(num_users, num_logs)

    print("\nSample Logs (First 5 Records)")
    print_separator()
    for log in logs[:5]:
        print(log)
    print_separator()

    start = time.time()
    totals = total_time_per_user(logs)
    end = time.time()

    print_totals(totals)
    print(f"Time taken: {(end - start):.6f} seconds")

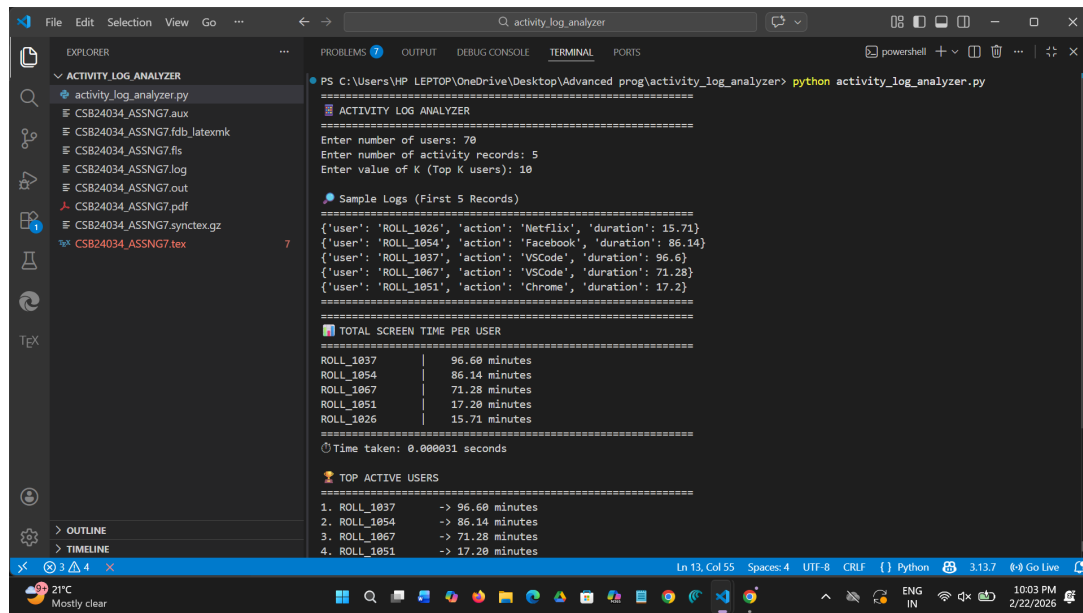
    start = time.time()
    top_users = most_active_users(logs, k)
    end = time.time()

    print_top_users(top_users, totals)
    print(f"Time taken: {(end - start):.6f} seconds")

    actions = unique_actions(logs)
    print_unique_actions(actions)

```

4 Program Execution Outputs



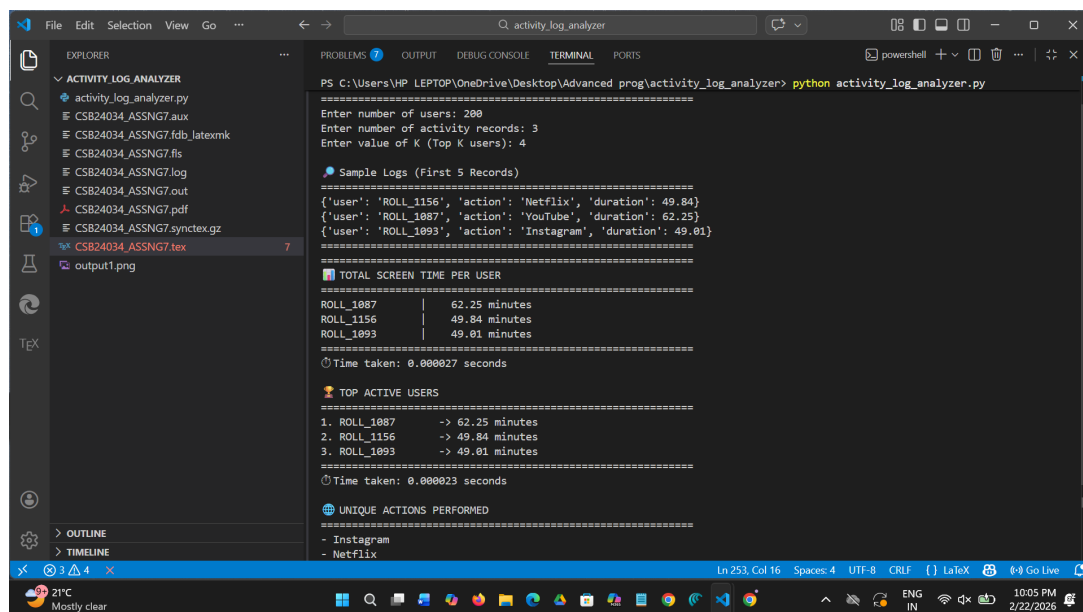
```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\activity_log_analyzer> python activity_log_analyzer.py
=====
ACTIVITY LOG ANALYZER
=====
Enter number of users: 70
Enter number of activity records: 5
Enter value of K (Top K users): 10

Sample Logs (First 5 Records)
=====
{'user': 'ROLL_1026', 'action': 'Netflix', 'duration': 15.71}
{'user': 'ROLL_1054', 'action': 'Facebook', 'duration': 86.14}
{'user': 'ROLL_1037', 'action': 'VSCode', 'duration': 96.6}
{'user': 'ROLL_1067', 'action': 'VSCode', 'duration': 71.28}
{'user': 'ROLL_1051', 'action': 'Chrome', 'duration': 17.2}
=====

TOTAL SCREEN TIME PER USER
=====
ROLL_1037 | 96.60 minutes
ROLL_1054 | 86.14 minutes
ROLL_1067 | 71.28 minutes
ROLL_1051 | 17.20 minutes
ROLL_1026 | 15.71 minutes
=====
Time taken: 0.000031 seconds

TOP ACTIVE USERS
=====
1. ROLL_1037 -> 96.60 minutes
2. ROLL_1054 -> 86.14 minutes
3. ROLL_1067 -> 71.28 minutes
4. ROLL_1051 -> 17.20 minutes
=====
```

Figure 1: Execution 1: Total Screen Time and Top K Users



```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\activity_log_analyzer> python activity_log_analyzer.py
Enter number of users: 200
Enter number of activity records: 3
Enter value of K (Top K users): 4

Sample Logs (First 5 Records)
=====
{'user': 'ROLL_1156', 'action': 'Netflix', 'duration': 49.84}
{'user': 'ROLL_1087', 'action': 'YouTube', 'duration': 62.25}
{'user': 'ROLL_1093', 'action': 'Instagram', 'duration': 49.01}
=====

TOTAL SCREEN TIME PER USER
=====
ROLL_1087 | 62.25 minutes
ROLL_1156 | 49.84 minutes
ROLL_1093 | 49.01 minutes
=====
Time taken: 0.000027 seconds

TOP ACTIVE USERS
=====
1. ROLL_1087 -> 62.25 minutes
2. ROLL_1156 -> 49.84 minutes
3. ROLL_1093 -> 49.01 minutes
=====
Time taken: 0.000023 seconds

UNIQUE ACTIONS PERFORMED
=====
- Instagram
- Netflix
=====
```

Figure 2: Execution 2: Different Input Values

```

PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced_prog\activity_log_analyzer> python activity_log_analyzer.py
{'user': 'ROLL_1532', 'action': 'Instagram', 'duration': 49.47}
{'user': 'ROLL_1373', 'action': 'WhatsApp', 'duration': 56.85}
{'user': 'ROLL_1148', 'action': 'WhatsApp', 'duration': 7.66}
{'user': 'ROLL_1836', 'action': 'Telegram', 'duration': 37.8}
{'user': 'ROLL_1816', 'action': 'Netflix', 'duration': 38.27}

=====
TOTAL SCREEN TIME PER USER
=====
ROLL_1373      | 56.85 minutes
ROLL_1532      | 49.47 minutes
ROLL_1816      | 38.27 minutes
ROLL_1836      | 37.80 minutes
ROLL_1148      | 7.66 minutes
=====
Time taken: 0.000028 seconds

=====
TOP ACTIVE USERS
=====
1. ROLL_1373    -> 56.85 minutes
2. ROLL_1532    -> 49.47 minutes
=====
Time taken: 0.000015 seconds

=====
UNIQUE ACTIONS PERFORMED
=====
- Instagram
- Netflix
- Telegram
- WhatsApp
=====

```

Figure 3: Execution 3: Unique Actions and Execution Time

5 Time Complexity Analysis

- Log generation: $O(n)$
- Total time calculation: $O(n)$
- Sorting for Top K: $O(m \log m)$
- Unique action extraction: $O(n)$

Where:

- n = number of logs
- m = number of users

6 Space Complexity Analysis

The space complexity of the Activity Log Analyzer program is determined by the memory required to store input data and intermediate results.

Let:

- n = Number of activity logs
- m = Number of unique users
- a = Number of unique actions

6.1 Storage of Logs

The program generates and stores all activity logs in a list:

```
logs = generate_logs(num_users, num_logs)
```

Since each log entry is stored as a dictionary inside a list of size n , the space required is:

$$O(n)$$

6.2 Total Time Per User Dictionary

The function `total_time_per_user()` creates a dictionary that stores total time for each unique user. In the worst case, each log belongs to a different user.

Thus, the space required is:

$$O(m)$$

Since $m \leq n$, this becomes:

$$O(n)$$

6.3 Sorted Users List

The `most_active_users()` function creates a sorted list of users based on total activity time. This list stores at most m users.

$$O(m)$$

Again, in the worst case:

$$O(n)$$

6.4 Unique Actions Set

The `unique_actions()` function creates a set of unique actions. Since the number of possible actions is fixed and small (e.g., 8 predefined applications), this requires constant space:

$$O(a) = O(1)$$

6.5 Overall Space Complexity

Combining all components:

$$O(n) + O(m) + O(a)$$

Since $m \leq n$ and a is constant, the overall space complexity simplifies to:

$$\boxed{O(n)}$$

Therefore, the program requires linear space proportional to the number of activity logs.

7 Conclusion

The Activity Log Analyzer successfully demonstrates data processing using Python functional programming techniques and built-in data structures. The program efficiently computes user activity statistics and provides well-formatted analytical output.